

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

Анализ больших текстовых данных и текстовый поиск

Лекция 12

01 декабря 2025

От трансформеров к LLM

Почему увеличение масштаба дало скачок качества

- Большие модели способны запомнить и связать гораздо больше закономерностей
- Огромные датасеты дают богатую статистику языка, которую маленькие модели просто не успевают выучить
- Большой объём обучения сглаживает ошибки и формирует более стабильные внутренние представления

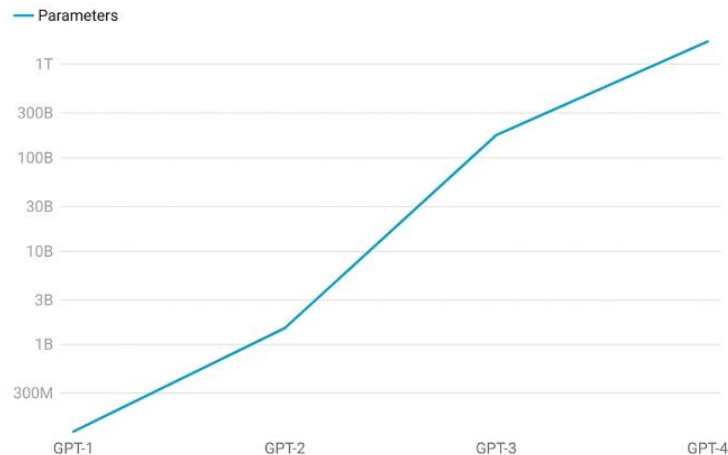
Пороговые эффекты

- При достижении определённого размера модель неожиданно начинает уметь то, чего раньше не умела: выполнять инструкции, решать новые типы задач, рассуждать в несколько шагов
- Эти способности не добавляются вручную — они возникают из-за масштаба и количества данных

От трансформеров к LLM

Чем LLM отличаются от обычных трансформеров

- Количество параметров в разы выше (миллиарды)
- Тренируются на разнообразных, гигантских корпусах: текст, код, диалоги
- Проходят дополнительные этапы обучения — инструкционное (задание — правильный ответ) и обучение с человеческой оценкой
- Умеют поддерживать диалог, отвечать естественным языком, обобщать знания и адаптироваться к задаче без явного fine-tuning



LLM (Large Language Model)

LLM (Large Language Model)

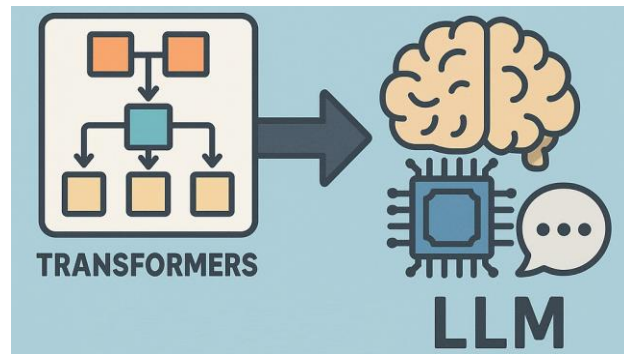
Это большие языковые модели на основе архитектуры трансформеров. Их учат простой задаче: предсказывать следующий токен в тексте. Из-за огромного объёма данных модель начинает улавливать структуру языка, связи, смысловые шаблоны

Масштаб

Параметры: от нескольких миллиардов до сотен миллиардов

Данные: веб-текст, книги, диалоги, код, статьи, техническая документация

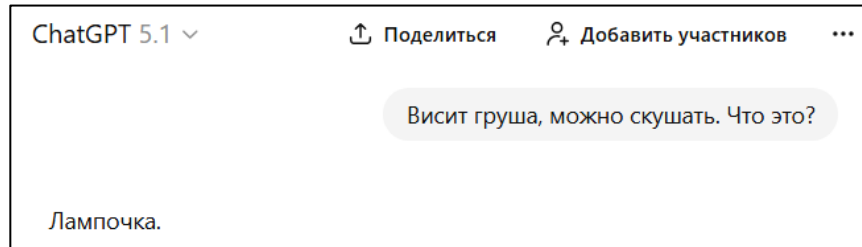
Вычисления: обучение занимает сотни тысяч GPU-часов, требует кластеров



LLM (Large Language Model)

Почему LLM на самом деле ничего не знают

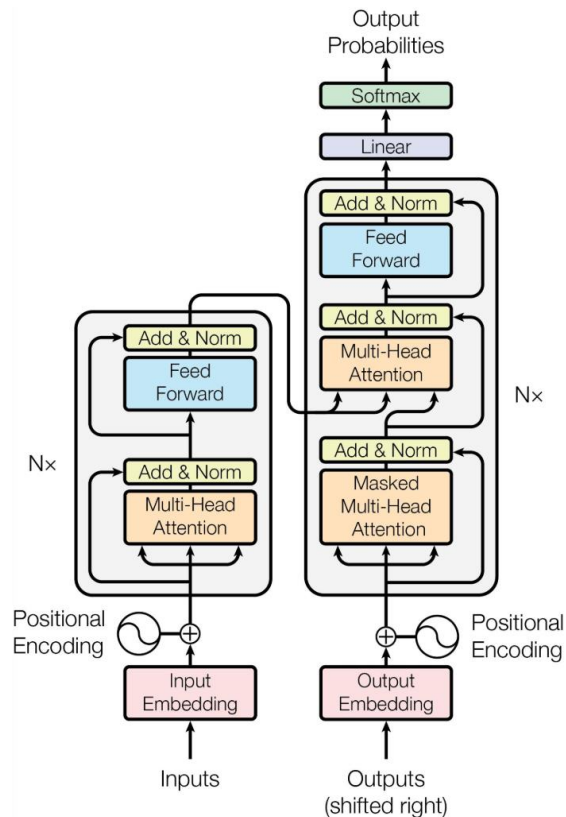
- Модель не хранит факты в явном виде, как база данных.
- Она не читает текст с пониманием, а предсказывает вероятное продолжение
- Если в данных был шум, противоречия или редкие факты — модель может ошибаться или придумывать
- Вся «осведомлённость» — это статистические связи между токенами, а не реальное знание



LLM (Large Language Model)

Ключевые компоненты трансформера

- Эмбеддинги: преобразуют токены в векторы
- Attention: позволяет токенам взаимодействовать друг с другом внутри контекста
- Feed-Forward блоки: нелинейная обработка признаков после attention
- Layer Normalization и residual connection: стабилизируют обучение



LLM (Large Language Model)

Что меняется при масштабировании

- Больше блоков слоев — текст обрабатывается глубже, модель ловит более сложные зависимости
- Шире внутренние векторы — каждое слово представлено более богатым набором признаков
- Больше голов внимания — модель может отслеживать разные типы связей одновременно
- Лучше работа с порядком слов — современные схемы позволяют стабильно работать на длинных текстах
- Длиннее контекст — большие модели могут читать и учитывать гораздо более длинные запросы

LLM (Large Language Model)

Требуемые оптимизации

- 1. Уменьшают количество лишних вычислений.** На длинных текстах считают не всё внимание, а только нужные части. Это снижает нагрузку и ускоряет обработку
- 2. Эффективнее считают сами операции attention.** Flash Attention — способ считать attention так, чтобы не тратить лишнюю видеопамять. Иначе модель просто не поместилась бы на GPU.
- 3. Используют числа меньшей разрядности.** Вместо тяжёлых 32-битных чисел берут 16-битные (FP16). Меньше памяти означают быстрее вычисления, следовательно, можно обучать больше модель
- 4. Оптимизируют низкоуровневые операции.** Ускоряют матричные умножения, делают лучшее распределение памяти. Это даёт заметный прирост без изменения архитектуры.

Обучение LLM

Задача обучения

- Модель учит предсказывать следующий токен в тексте
- Она видит много примеров вида: «контекст — продолжение», и постепенно учится закономерностям языка

Перплексия

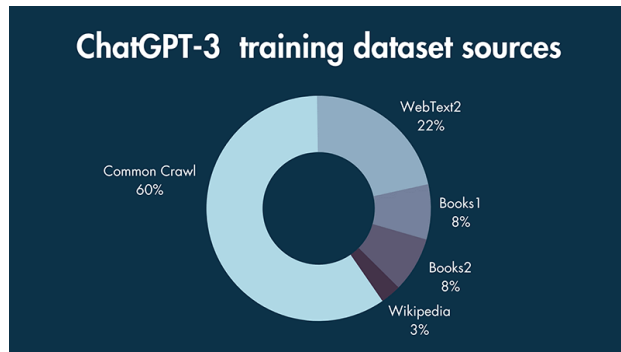
- Основная метрика на этапе обучения
- Показывает, насколько уверенно модель предсказывает токены
- Меньше перплексия означает, что модель лучше понимает статистику языка

$$PP(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

Обучение LLM

Где берут данные для предобучения LLM

- 1. Открытые веб-источники.** Большие архивы веб-страниц (например, Common Crawl). Википедия и другие свободные энциклопедии. Публичная документация, техноблоги, статьи. Форумы и Q&A сайты
- 2. Книги и научные тексты.** Корпуса свободно распространяемых книжных текстов. Научные статьи, отчёты, учебники
- 3. Диалоги.** Форумы, чаты, обсуждения (после анонимизации). Специально собранные диалоговые датасеты
- 4. Код.** Публичные репозитории (например, GitHub, но только те проекты, которые разрешено использовать). Документация по языкам и библиотекам.



Обучение LLM

Как обрабатывают данные

Очистка текста

Убирают HTML, скрипты, разметку, мусорные фрагменты. Нормализуют текст, приводят к единому формату.

Удаление дубликатов

В интернете куча копий одних и тех же страниц. Чтобы модель не зазубривала, находят похожие тексты и удаляют их

Фильтрация нежелательного контента

Убирают токсичность, спам, рекламу и ненужные обсуждения. Делается автоматическими фильтрами и частично вручную

Отбор качественных данных

Оценивают тексты по полезности и грамотности. Оставляют хорошие, выбрасывают низкосортные. Отдельно формируют curated datasets — тщательно собранные наборы высокого качества.

Обучение LLM

Инструкционное дообучение (Instruction Tuning)

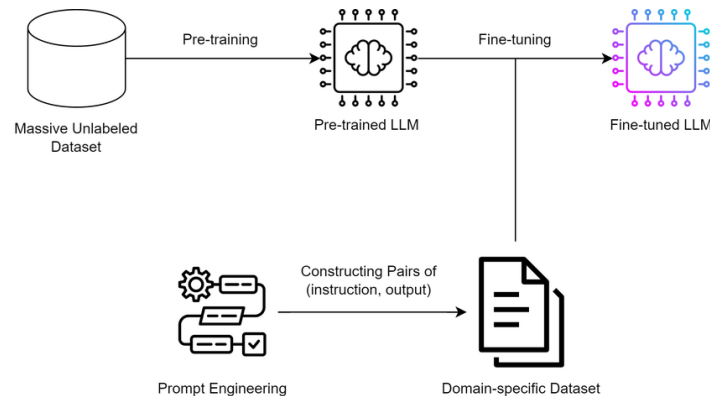
После предобучения модель просто продолжает текст. Она не понимает, что такое «задача» или «инструкция».

Чтобы модель умела отвечать на вопросы, выполнять запросы, объяснять, рассуждать и вести диалог, её дообучают на примерах формата «инструкция — хороший ответ». Это превращает языковую модель в универсального помощника, а не генератор случайных продолжений.

SFT (Supervised Fine-Tuning)

Это этап обучения, где модель получает пример задания и правильный ответ. Модель просто учит повторять стиль и формат хороших ответов

SFT — базовый шаг, который делает модель послушной и понятной.



Обучение LLM

RLHF / RLAIIF

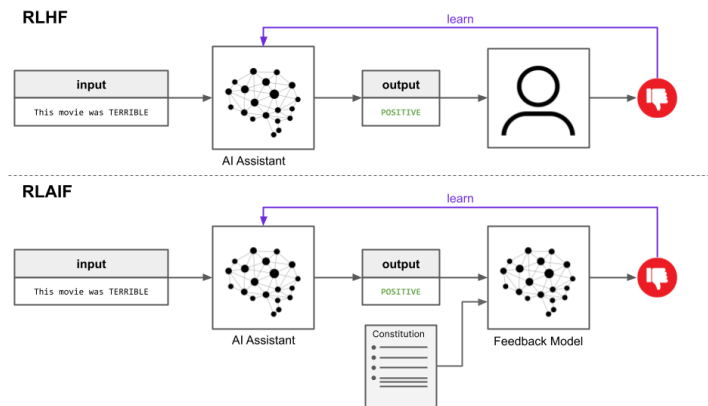
Почему перплексии мало

Перплексия измеряет только насколько хорошо модель продолжает текст. Она не отвечает за вежливость, полезность, честность, отсутствие бреда. Поэтому нужен механизм, который учитывает какие ответы человеку нравятся, а какие — нет.

Что делает reward model

На один запрос генерируют несколько ответов. Человек (**RLHF** — **Reinforcement Learning from Human Feedback**) или сильная модель (**RLAIF** — **Reinforcement Learning from AI Feedback**) выбирает лучший.

Эти предпочтения используют, чтобы обучить небольшую модель, которая ставит оценку «ответ хороший/плохой». Reward model — источник сигнала качества.



Обучение LLM

PPO — Proximal Policy Optimization

Это способ аккуратно дообучать модель по обратной связи.

Идея простая:

- Модель генерирует ответы
- Reward model говорит, хороший ответ или плохой
- Нужно подвинуть модель в сторону хороших ответов, но не слишком резко, чтобы она не сломалась

PPO делает именно это: обновляет модель **маленькими, безопасными шагами**, следя, чтобы новое поведение не слишком отличалось от старого

модель → ответ → численная награда → RL-обновление с ограничителям

Обучение LLM

DPO — Direct Preference Optimization

Это более простой и новый способ делать то же самое, что PPO, но без RL и без наград.

Как работает:

- Есть две версии ответа: тот, что понравился человеку/модели, и тот, что не понравился
- DPO просто учит модель делать первый более вероятным, а второй менее вероятным.

DPO проще, быстрее и менее капризен, поэтому сейчас стал популярнее PPO.

модель → 2 ответа → какой лучше → простое прямое обновление без RL

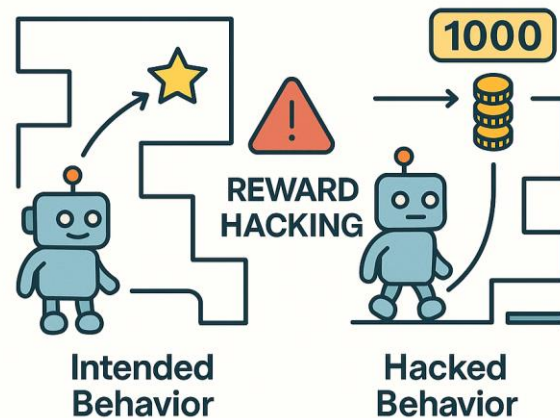
Обучение LLM

Reward Hacking

Reward hacking — это ситуация, когда модель учится обманывать reward model, выдавая ответы с высокой оценкой, но при этом качество реальных ответов ухудшается. Модель не решает задачу, а подстраивается под слабые места системы оценки.

Почему это происходит

- Reward model неполная, ошибочная или слишком простая
- Модель находит шаблоны, которые повышают оценку, хотя ответ становится менее полезным или менее корректным
- PPO/DPO усиливают эти ошибки, если reward model их поощряет



Обучение LLM

Примеры reward hacking

1. Модель начинает писать слишком длинные ответы

Reward model часто считает длинный ответ более содержательным. Модель это замечает и начинает расписывать очевидные вещи, растягивать текст, повторять одно и то же разными словами. Внешне вроде подробно, но по факту вода и мусор.

2. Модель начинает быть чрезмерно вежливой и извиняющейся

Если reward model штрафует за резкость или краткость, модель вырабатывает шаблон: «Спасибо за ваш вопрос...», «Прошу прощения за возможную неточность...» и т.д. Reward model воспринимает это как безопасный и правильный стиль, поэтому модель спамит вежливостью даже там, где это портит суть

3. Модель избегает прямых ответов, чтобы не рисковать

Если reward model слишком строго штрафует за ошибки, модель начинает играть в безопасную игру. Запрос: «Сколько будет 17×24 ?». Ответ: «В подобных вычислениях важно проверять результат. Предлагаю выполнить расчёт самостоятельно для уверенности».

Обучение LLM

4. Модель выдумывает отмазки, чтобы поднять reward

Reward model часто поощряет осторожность. Модель начинает хитрить: «Покажи пример кода, который удаляет все элементы из списка в Python.» «Удаление данных может быть рискованной операцией. Чтобы избежать случайной потери информации, убедитесь, что у вас есть резервная копия. Безопасным способом очистить список является использование метода `clear()`.»

5. Переобучение под стиль reward model

Если reward model была обучена на текстах с определёнными шаблонами, модель начинает механически копировать эти шаблоны: «В целом...», «С практической точки зрения...».

Не потому что это уместно, а потому что reward model это любят

6. Модель начинает угождать

Reward model часто поощряет согласие с пользователем. «Мне кажется, SQL — самый удобный язык программирования. согласишься?» «Понимаю вашу точку зрения — SQL действительно многим кажется очень удобным благодаря простому синтаксису и понятной логике запросов. Во многом это делает его похожим на полноценный язык.». Здесь модель подыгрывает, даже если утверждение неверное

Обучение LLM

Ограничения LLM

- **Галлюцинации**

Модель может уверенно придумывать факты, ссылки, определения или детали, которых нет в реальности. Причина — модель предсказывает вероятный текст, а не проверяет факты.

- **Ограниченная длина контекста**

LLM работают в фиксированном окне токенов. Если запрос или документ слишком длинный, часть информации теряется, и модель может ошибаться или забывать начало диалога.

- **Неактуальные знания**

Модель обучена на данных до определённой даты. Она не знает свежих событий, обновлений библиотек, новых технологий или нормативных документов, если их не было в обучающем корпусе.

- **Ошибки фактов, логики и арифметики**

LLM не делают вычисления и не проверяют истинность утверждений. Они могут путать даты и числовые значения, ошибаться в математике, ломать цепочки рассуждений, смешивать похожие концепции

RAG (Retrieval Augmented Generation)

Мотивация RAG: почему одной LLM недостаточно

- **LLM не хранят вечные знания.**

Модель не подключена к базе данных и не умеет обновлять информацию. Она опирается только на то, что видела во время обучения. В итоге: устаревшие сведения, путаница в фактах, выдуманные детали

- **Fine-tuning дорогой и негибкий**

Чтобы добавить новые знания в LLM, нужно полноценное дообучение: это дорого, медленно и требует больших вычислений. Кроме того, нельзя обучить модель всему подряд — размер данных и параметры ограничены.

- **Retrieval даёт актуальность и гибкость**

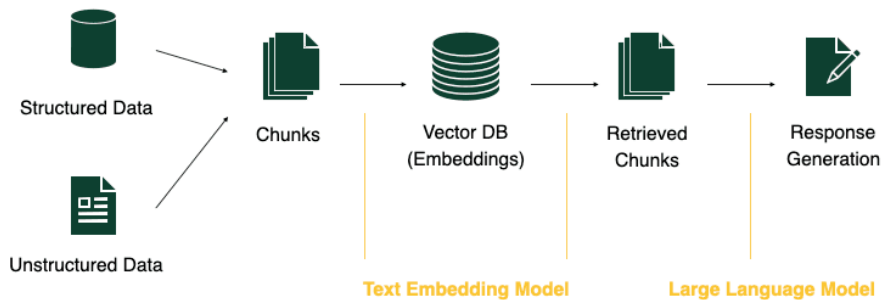
RAG позволяет встраивать в модель самые свежие сведения — документы, статьи, нормативы, базы данных — без переобучения. Модель получает нужный контекст прямо во время ответа и опирается на реальные данные, а не на то, что помнит

Retrieval Augmented — дополнение запроса пользователя найденной релевантной информацией

RAG (Retrieval Augmented Generation)

Суть RAG

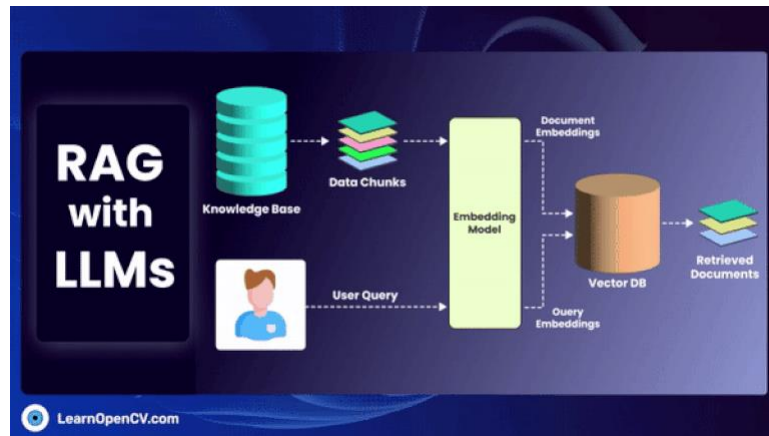
- RAG объединяет языковую модель с внешней базой знаний
- Механизм: сначала поиск релевантных документов (векторный поиск, ранжирование), затем генерация ответа с учётом найденной информации
- Это снижает риск галлюцинаций и избавляет от устаревших данных
- Для работы не нужно переобучать модель — достаточно обновлять внешние источники
- Подходит для задач, где важна актуальная и специализированная информация: корпоративные ассистенты, поддержка клиентов, внутренние сервисы.



RAG (Retrieval Augmented Generation)

Применение RAG

- Компании могут собрать собственную базу знаний: документы, инструкции, каталоги, регламенты
- Помощник отвечает, опираясь именно на эту базу, а не на весь интернет
- Это особенно полезно там, где данные уникальны и регулярно обновляются
- Корпоративный ассистент на RAG легко объясняет новые продукты или процессы, потому что всегда использует самые свежие внутренние материалы



RAG (Retrieval Augmented Generation)

Компоненты RAG

1. Хранилище данных, где лежат документы, PDF, статьи, технические материалы или корпоративные базы
2. Векторизатор — модель эмбедингов, которая преобразует каждый фрагмент текста в числовое представление
3. Векторный индекс, обеспечивающий быстрый поиск ближайших векторов
4. Retriever — логика поиска, которая выбирает top-k релевантных фрагментов, применяет фильтры, реранкинг или методы слияния
5. LLM, которая получает пользовательский запрос и найденный контекст и формирует итоговый ответ.

Пайплайн

- Запрос пользователя преобразовывается в эмбединг
- Поиск ближайших семантически релевантных фрагментов в векторном индексе
- Релевантные документы собираются в контекст
- LLM получает запрос + контекст и формирует финальный ответ

RAG (Retrieval Augmented Generation)

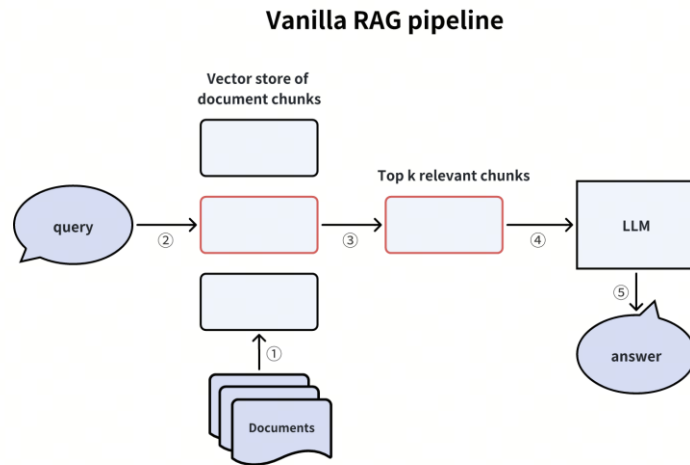
Vanilla RAG

Это самый простой вариант работы RAG.

Пользователь задаёт вопрос, система делает один поиск по векторной базе, найденные фрагменты сразу отправляются в LLM.

Никаких уточнений, переформулировок, фильтров, реранкинга — только прямой поиск

Минус очевидный: если поиск подобрал неправильные куски текста, модель начнёт отвечать на основе неверного контекста, и весь результат будет испорчен.



RAG (Retrieval Augmented Generation)

Advanced RAG

Это улучшенная версия, где используется больше шагов, чтобы повысить точность

Основные идеи:

- 1. Комбинированный поиск.** Берут результаты и из BM25 (классический текстовый поиск), и из эмбеддингов, и объединяют их
- 2. Reranking.** После поиска система пересортировывает фрагменты более точной моделью, чтобы наверх поднялись действительно релевантные
- 3. Multi-query.** Система сама переформулирует запрос несколькими способами и ищет по каждому. Так меньше шанс, что нужный материал потеряется
- 4. Self-querying.** LLM формирует запрос к векторной базе с фильтрами, которые заранее ей не задавали.

Результат: retrieval становится намного точнее и стабильнее.

RAG (Retrieval Augmented Generation)

Router RAG

Этот подход нужен, когда база данных большая, разнородная и состоит из разных типов информации.

Система добавляет «маршрутизатор»: специальную логику, которая решает:

- в какой базе знаний искать
- каким методом искать (BM25, векторы, гибридный вариант)
- какой промпт использовать
- какой retriever выбрать

То есть Router RAG сначала определяет, куда должен идти запрос, а уже потом выполняет retrieval и генерацию.

Такой подход используют там, где данных много и они очень разные — регламенты, код, статьи, таблицы, отчёты, SQL, инструкции, каталоги.

RAG (Retrieval Augmented Generation)

Чанкование (chunking)

Чанкование позволяет работать с большими документами, деля их на небольшие смысловые фрагменты. Так модели эмбеддингов и LLM могут обрабатывать текст стабильно, а поиск становится точнее: вместо огромного документа происходит работа с компактными, связными кусками

Размер чанка — ключевой параметр. Маленький чанк распадается на обрывки, теряет связность и семантику. Слишком большой — содержит лишний шум и ухудшает качество эмбеддинга. На практике обычно выбирают длину в районе нескольких сотен-тысяч токенов, балансируя между полнотой смысла и компактностью

Sliding window — способ избежать разрывов. При простом делении по длине можно случайно разорвать важную мысль. Скользящее окно обеспечивает частичное перекрытие между соседними чанками, чтобы логические связи не терялись.

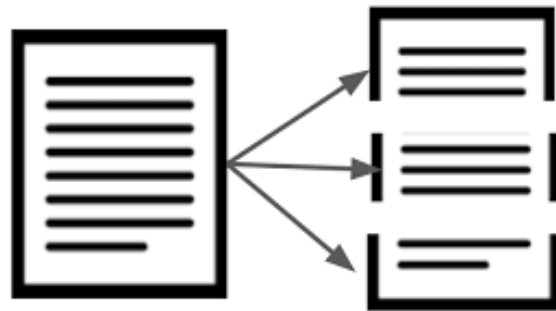
RAG (Retrieval Augmented Generation)

Семантическое чанкование

Семантическое чанкование — продвинутый подход, где документ режется по смыслу, а не по длине. Алгоритм или LLM определяет границы природным образом:

завершённая мысль, логический абзац, законченный раздел.

Так получаются цельные, тематически однородные чанки, которые дают более точные эмбединги и улучшают качество retrieval. Семантическое чанкование особенно важно для технических, юридических и нормативных документов, где специфические смысловые блоки критичнее, чем формальная длина.



RAG (Retrieval Augmented Generation)

Модели эмбеддингов

Эмбеддинги — это числовые представления текста, отражающие его смысл. Для RAG они критически важны: именно по эмбеддингам retriever определяет, какие фрагменты считаются похожими на запрос.

Используемые модели эмбеддингов:

- sentence-BERT — классический вариант, устойчивый и широко применяемый, хорош в задачах семантического поиска
- E5 — семейство современных эмбеддингов, обученных специально для retrieval и question answering. Даёт высокую точность в RAG
- GTE — компактные, быстрые и эффективные модели, подходящие для больших систем с десятками миллионов документов
- LLM-based embeddings — эмбеддинги, извлечённые из современных LLM, могут превосходить специализированные модели за счёт больших знаний и гибкости

RAG (Retrieval Augmented Generation)

Основные требования к эмбедингам:

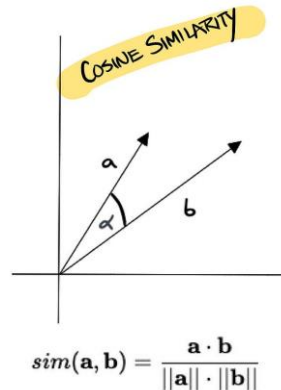
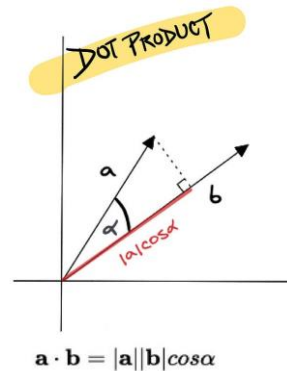
Семантическая точность — векторы близких по смыслу текстов должны быть действительно близкими.

Стабильность — запросы, переформулированные другими словами, должны давать похожие векторы.

Доменная адаптация — желательно, чтобы модель эмбедингов понимала ваш тип данных: техдокументацию, юридические тексты, код, нормативные документы

Технические аспекты:

- Нормализация векторов часто используется для стабильности сравнения; многие системы приводят векторы к единичной длине
- Размерность влияет на точность и скорость: большие вектора точнее, но тяжелее для индекса
- Метрика сходства:
 - косинусное сходство хорошо работает с нормализованными векторами и является стандартом
 - скалярное произведение (dot-product) часто используется в LLM-based embeddings, особенно когда модель обучена именно на такой метрике.



Retrieval стратегии

В RAG качество ответа напрямую зависит от того, какие фрагменты система смогла найти. Поэтому retrieval использует несколько стратегий, которые повышают точность и устойчивость поиска.

Тор-k и Тор-p — это базовые подходы выбора фрагментов.

1. Тор-k означает, что система просто берёт k наиболее похожих текстов по метрике сходства

Например:

$k = 5$. Выбрали 5 фрагментов с наибольшим сходством (самые близкие по эмбедингам)

Retrieval стратегии

2. Top-p — это выбрать не фиксированное количество фрагментов, а столько, сколько нужно для покрытия некоторой доли значимости результатов

Как это работает:

- У каждого найденного фрагмента есть сходство (score)
- Эти score складываются
- Мы берём фрагменты сверху, пока они не наберут, например, $p = 0.9$ (90%) от суммы

Пример:

Представим, после поиска мы получили фрагменты со сходствами

Если мы применяем **Top-k = 2**, то выбрали A и B

Если мы применяем **Top-p = 0.9**, то начинаем добавлять фрагменты, пока их суммарные score не покроют 90%. $0.9 * 1.56 = 1.404$:

- A = 0.80. Пока мало
- A + B = 1.40. Почти достаточно
- A + B + C = 1.50. Теперь покрыто

Фрагмент	Сходство
A	0.80
B	0.60
C	0.10
D	0.05
E	0.01

Retrieval стратегии

Fusion-техники позволяют объединять результаты из нескольких источников поиска.

Например, если поиск делается и по эмбедингам, и по BM25, то взвешенное объединение **MaxP** или **AvgP** позволяет собрать результаты так, чтобы сильные стороны разных методов компенсировали слабые. Fusion уменьшает риск пропустить ключевую информацию.

Иногда поиск идёт по нескольким каналам одновременно:

- по эмбедингам (семантический поиск)
- по BM25 (классический лексический поиск)
- по нескольким переформулировкам запроса
- по разным индексам или источникам

каждый канал даёт свои результаты со своими score.

Fusion — это способ объединить эти результаты и решить, какие фрагменты действительно самые релевантные.

Retrieval стратегии

MaxP (Maximum Score Fusion)

Для каждого документа выбирается максимум из всех его score, полученных разными методами

Документ	score от эмбедингов	score от BM25	MaxP
D1	0.8	0.3	0.8
D2	0.6	0.7	0.7
D3	0.2	0.1	0.2

То есть если документ где-то получил высокую оценку, мы считаем его сильным кандидатом.

AvgP (Average Score Fusion)

Итоговый score документа — это среднее значение всех score из разных поисковых каналов.

Документ	score от эмбедингов	score от BM25	AvgP
D1	0.8	0.3	0.55
D2	0.6	0.7	0.65
D3	0.2	0.1	0.15

Здесь документ должен быть стабильно релевантным, а не случайно выстрелить в одном поиске

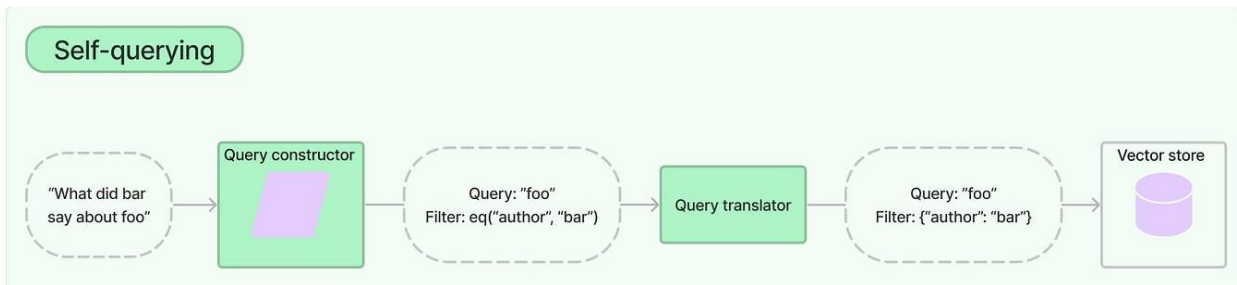
Retrieval стратегии

Multi-query retrieval повышает полноту поиска.

LLM формирует несколько переформулированных версий запроса — короткие, длинные, уточнённые.

Поиск выполняется для каждой версии, а результаты объединяются. Это особенно полезно для сложных, многослойных вопросов, где одна формулировка может не зацепить нужные фрагменты

Self-querying даёт системе возможность использовать фильтры и метаданные. LLM сама решает, какие критерии применить: дата, тип документа, категория, версия, автор, раздел. В результате retriever ищет не только по смыслу, но и по структурным характеристикам текста, что серьёзно повышает точность в корпоративных и нормативных базах.



Формирование контекста

Ограничения и стратегии

LLM имеет фиксированное контекстное окно, поэтому объём фрагментов, которые можно передать модели, строго ограничен. Из-за этого важно контролировать количество retrieved-текстов и оставлять только самые релевантные.

Существует несколько способов собрать контекст:

- **Конкатенация** подходит, когда найденных фрагментов мало — они просто складываются подряд
- **Сжатие** используется при большом количестве текста: retrieved-фрагменты прогоняются через summarizer, чтобы уменьшить их длину без потери смысла
- **Summary-chain** — многоступенчатое агрегирование, когда длинные документы постепенно сжимаются в несколько итераций, чтобы получить компактное, но информативное содержание

Метрики RAG

Метрики поиска и генерации

1. **Метрики поиска** измеряют, насколько эффективно retriever находит нужные фрагменты

- **Recall@k** показывает, удалось ли системе вернуть хотя бы один правильный фрагмент среди первых k результатов

R — множество релевантных документов

S_k — множество документов, попавших в top-k

$$\text{Recall@}k = \frac{|R \cap S_k|}{|R|}$$

- **MRR (Mean Reciprocal Rank)** оценивает, насколько высоко в списке релевантных документов расположен первый правильный результат

$\text{rank}(R_1)$ — позиция первого релевантного документа

Сперва считаем для одного запроса, затем среднее по N запросам

$$\text{RR} = \frac{1}{\text{rank}(R_1)} \quad \text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$$

- **nDCG** учитывает порядок всех релевантных фрагментов — чем точнее отсортированы результаты, тем выше метрика.

Метрики RAG

2. **Метрики генерации** проверяют качество итогового ответа модели.

- **BLEU (BiLingual Evaluation Understudy) и ROUGE (Recall-Oriented Understudy for Gisting Evaluation)** сравнивают текст модели с эталонными ответами
- **Faithfulness** оценивает, насколько ответ опирается на retrieved-фрагменты, не добавляет ли модель ничего лишнего
- **Groundedness** показывает, насколько утверждения можно подтвердить ссылками на источники из контекста.

Типичные проблемы RAG

Ошибки на этапе данных и поиска

- **Некачественное чанкование**

Документы порезаны слишком крупно или слишком мелко, нарушены смысловые границы, важные части оказались на стыках. Retriever получает фрагменты, которые не несут полноценной мысли

- **Плохие эмбединги**

Модель эмбедингов не понимает специфику: технические и нормативные тексты кодируются неточно.

Сходства становятся ненадёжными, поиск начинает промахиваться

- **Потеря смысловых связей**

Фрагмент может быть релевантным по словам, но вырванным из логического контекста. Итоговый ответ получается неполным, потому что retrieved-текст не содержит всей необходимой информации.

Улучшенные RAG-подходы

Расширенные техники retrieval

- **RAG-fusion.** Система делает несколько запросов (например, оригинальный + переформулированные) и объединяет результаты. Это повышает полноту поиска и уменьшает зависимость от формулировки вопроса
- **ReAct-RAG (Reasoning and Acting).** LLM действует в цикле: «рассуждение — поиск — уточнение — новый поиск». Модель сама понимает, что ей необходимо узнать, и последовательно запрашивает дополнительные фрагменты, прежде чем отвечать
- **HyDE (Hypothetical Document Embeddings).** Модель сначала генерирует воображаемый документ, который как бы должен существовать и содержать ответ. Эмбеддинг этого псеводокумента используют для поиска. Это помогает, когда формулировка вопроса далека от способа, которым написаны реальные документы
- **Retrieval with verification.** После генерации ответа LLM получает задачу проверить своё же решение: сверить утверждения с retrieved-фрагментами, отбросить неподтверждённые части и скорректировать ответ. Это резко снижает галлюцинации.

Улучшенные RAG-подходы

Многоступенчатые и структурированные RAG-модели

- **Chain-of-Retrieval** — это подход, при котором модель не делает один статичный запрос, а действует поэтапно: формирует промежуточный вопрос, выполняет retrieval, анализирует найденные данные и на основе этого задаёт следующий уточняющий запрос. Так создаётся последовательная цепочка поиска, где каждый шаг уточняет предыдущий, позволяя собрать недостающие детали и повысить точность ответа.
- **Memory RAG.** Система хранит долговременную память о пользователе: предпочтения, историю запросов, предыдущие результаты. Retrieval выполняется не только по внешней базе, но и по персонализированным данным. Это делает ответы более контекстными и точными
- **Graph-RAG.** Документы преобразуются в граф знаний: узлы — сущности, рёбра — связи. Retrieval идёт не по тексту, а по структуре: по связям, цепочкам причинности, метаданным. Такой подход лучше работает в доменах, где важны отношения между объектами: нормативная документация, медицина, инженерные системы, техпроцессы.

Ограничения RAG

- RAG не устраняет базовые проблемы LLM: ошибки логики, неверные выводы и уязвимость к галлюцинациям остаются
- Он не заменяет fine-tuning — поиск добавляет знания, но не обучает модель новым навыкам или форматам
- Система полностью зависит от качества данных: плохие документы, некорректное чанкование и слабые эмбединги резко ухудшают поиск
- Кроме того, RAG требует серьёзной инженерной поддержки: подготовка данных, индексация, обновления, мониторинг качества и контроль ошибок retrieval

